

testA

Lazar Nagulov

June 5, 2025

Contents

1	Introduction	3
2	Getting Started	4
2.1	Prerequisites	4
2.2	Cloning the Project	4
2.3	Building the Project	4
2.4	Running Tests	4
3	Features	5
3.1	Templates	5
3.1.1	Inheritance	5
3.1.2	Referencing	6
3.2	Enumerations	7
3.3	Data types	8
3.3.1	Lists	8
3.3.2	Type constraints	9
3.4	String patterns	10
3.5	Attributes	12
3.6	Resources	13
3.7	Generate	14
A	EBNF Grammar	15
B	Changelog	15

1 Introduction

TestA is a domain-specific language designed for generating structured test data in formats like JSON, CSV, and XML. It provides a simple, C-like syntax for defining templates, resources, enums, and generation rules, enabling quick and expressive data mockups for testing, prototyping, or seeding.

```
@output csv {
    header = true;
    quote = true;
    delimiter = ",";
}
@output_path "./example.csv";

type Grade = int [range=6..=10];
type GradeList = [Grade] [count=0..=50];

enum Role {
    User => 10;
    Admin => 2;
    Other;
};

template User {
    id = $uuid();
    name = string;
    role = Role;
}

template Student : User {
    year = int [range=1..=4];
    grades = GradeList;
}

@generate User [10];
```

Listing 1: testA example

2 Getting Started

This section describes how to install the necessary tools and set up the project locally.

2.1 Prerequisites

To build and run this project, you must have the Rust toolchain installed, which includes the Rust compiler as well as Cargo, Rust's package manager and build system. To install Rust and Cargo, run the following command in your terminal:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

After installation, restart your terminal and verify the installation with:

```
rustc --version  
cargo --version
```

2.2 Cloning the Project

Use `git` to clone the repository:

```
git clone https://github.com/lazarnagulov/testa.git  
cd testa
```

2.3 Building the Project

To compile the project, run:

```
cargo build
```

To run the project:

```
cargo run -- ./examples/01_anonymous_generate.testa
```

2.4 Running Tests

To run the project's test suite:

```
cargo test
```

3 Features

3.1 Templates

```
template <name> [: <parent_name>] {  
    [override] <name1> = <expr>;  
    [override] <name1> = <expr>;  
    ...  
}
```

Listing 2: Template declaration

Templates are blueprints for creating reusable data structures or types that are used in a "generate" expression. Each template has a name and a set of fields, where each field is a pair consisting of a string (the field name) and an expression (which defines the field's value). By using data types and enumerations within these expressions, you can add randomness to the values generated by the template.

```
template User {  
    id = int;  
    name = string;  
    age = int;  
}
```

Listing 3: Example template in testA

3.1.1 Inheritance

```
template User { age = int [range=0..=100]; }  
template Student : User {  
    year = int [range=1..=4];  
    override age = int [range=19..=30];  
}
```

Listing 4: Inheritance example

Templates can be created from existing templates through a process known as inheritance (Listing 4). A child template inherits all the fields from its parent template and has the option to override any specific field by using the "override" keyword and specifying the field name.

While using this keyword is not mandatory, incorrect usage may lead to a warning. For example, Listing 5 will trigger a warning because the field "age" is not overridden in the child template. It's important to note that all repeated fields are overridden by default.

```
template User {
    id = string;
    //age = int [range=0..=100];
}
template Student : User {
    year = int[range=1..=4];
    // User template does not have 'age' field!
    override age = int[range=19..=30];
}
```

Listing 5: Override example

3.1.2 Referencing

3.2 Enumerations

```
enum <name> {  
    <variant1> [=] <expr:int>;  
    <variant2> [=] <expr:int>;  
    ...  
}
```

Listing 6: Enumeration declaration

The "testA" enumerations include a list of names and their corresponding weights, which indicate the likelihood of randomly generating a variant. Weights are always integers, with a default value of one if not specified.

```
enum Role {  
    User => 8;  
    Admin; // Default weight is 1  
    Developer => 3;  
}
```

Listing 7: Enumerations in testA

As shown in listing 8, enumerations can be used as types in templates. Their evaluated values are always represented as strings

```
template User {  
    id = int;  
    name = string;  
    role = Role;  
}
```

Listing 8: Template with enumeration field

3.3 Data types

Unlike other programming languages, such as C or Java, testA data types represent random values in expressions. There are four fundamental data types available: integers (ints), floating-point numbers (floats), strings, and booleans.

```
template User {  
    // Generates random string  
    name = string;  
    // Generates random integer  
    age = int + 18;  
    // Generates random float  
    salary = float + 300.0;  
    // Generates random boolean  
    is_registered = !bool;  
}
```

Listing 9: Template with data types

As shown in Listing 9, these types can interact with one another as well as with literals. Additionally, fundamental data types can be extended by introducing constraints or creating lists of these types.

3.3.1 Lists

```
[<type>][<constraint1>, <constraint2>...]
```

Listing 10: List type

A list is a container that can hold various data types, including other lists. By default, a list generates a random number of items, ranging from 0 to 16. However, this number can be adjusted using the "count" constraint. Further details about constraints can be found in the next section.

```
// Example 1: List.  
[int][count=5..15]  
// [1, -5, 6, -7, 8, 18]  
// Example 2: Multidimensional list.  
[[int][range=1..=5]][count=1..5][count=3..=5]  
// [[1,2,3],[5,2,1,3],[5],[4,5]]
```

Listing 11: List examples

In Examples from Listing 11, two lists are declared. Second list can be simplified by utilizing type constraints.

3.3.2 Type constraints

```
type <name> = <type> [<constraint1>, <constraint2>...];  
type <name> = extend <type> with [<constraint1>, ...];
```

Listing 12: Constrained type declaration

The previous example (Listing 9) does not depict a realistic individual, as the fields for age and salary could potentially contain negative values. To address this issue, we can implement type constraints to prevent such errors. These constraints can be easily added by enclosing them in brackets [] after the type name (Listing 13), ensuring that the data conforms to the expected parameters.

```
template User { age = int [range=0..=100]; }  
template Student : User {  
    override age = int [range=19..=100];  
    grades = [int [range=6..=10]] [count=0..=46];  
}
```

Listing 13: Templates with constrained data types

To improve clarity and minimize redundancy, types may be effectively declared and reused (Listings 14 and 15).

```
type Age = int [range=0..=100];  
type StudentAge = extend Age with [min=19];  
type Grade = int [range=6..=10];  
type GradeList = [Grade] [count=1..=46];  
  
template User { age = Age; }  
template Student : User {  
    override age = StudentAge;  
    grades = GradeList;  
}
```

Listing 14: Templates with user-defined data types

```

type IntList = [int][count=1..=5];
type IntMatrix = [IntList][count=1..=5];
type IntTensor = [IntMatrix][count=1..=5];
// same as
type IntTensor
    = [[[int][count=1..=5]][count=1..=5]][count=1..=5];

```

Listing 15: Templates with user-defined lists

Name	Compatible types	Input	Description
range	int, float	Inclusive or Exclusive range	Sets upper and lower bound.
min	int, float	int	Sets lower bound.
max	int, float	int	Sets upper bound.
multiple_of	int	int	Number is multiple of input.
bias	bool	float[0, 100]	Sets chance of being true.
count	list	Inclusive, Exclusive range or int	Sets number of items in list

Table 1: Data type constraints

3.4 String patterns

String patterns define the structure of a string. In these patterns, characters wrapped in \$ represent placeholders where random values will be generated. Every other character is treated as literal. Placeholders are:

- \${a} generates a random lowercase letter (a-z)
- \${A} generates a random uppercase letter (A-Z)
- \${#} generates a random digit (0-9)

Examples (Listing 16) can generate "pattern: abcABCD123"

```

string_pattern "pattern: ${aaaAAAA###}";

```

Listing 16: String pattern example

Patterns can be utilized directly within the template, as illustrated in Listing 17. They can also be simplified using the repeat syntax [<integer or range>].

```
template User {  
    id = int;  
    name = string_pattern "${Aa[4]}"; // same as Aaaaa  
    email = string_pattern "${a[1..=5]#[0..=4]}@${a[4]}.com"  
}
```

Listing 17: String pattern in template example

3.5 Attributes

Attributes attach metadata to data types, enumerations, templates, fields, and other entities. They can change the default entity behavior; for instance, marking the template as ‘abstract’ prevents it from being generated.

```
# [private]
type Age = int [range=18..=36];

# [public]
enum Role { Guest => 7; Member => 5; Moderator => 2; }

# [abstract]
template BaseUser {
  # [primary_key]
  id = string_pattern "ID_${#[3]}";
  role = Role;
}

template User : BaseUser {
  # [required]
  name = string_pattern "${Aa[1..=15]}";

  # [required]
  # [nullable]
  nickname = string;

  # [required]
  age = Age;
}
```

Listing 18: Attributes example

3.6 Resources

3.7 Generate

A EBNF Grammar

todo...

B Changelog

- v0.1 - Added csv template generation (MVP).
- v0.2 - Added support for enumerations and templates with inheritance.
- v0.3 - Introduced string patterns and attributes.